UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026

INFORME DEL DEBER PRÁCTICO

I. PORTADA

Tema:	Deber - Arboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero – B
Alumnos participantes:	Poaquiza Ayme Gilberth Joel
Asignatura:	ESTRUCTURA DE DATOS
Docente:	Ing. Jose Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

Objetivos

General:

Implementar en C++ un Árbol Binario de Búsqueda (BST) para organizar los empleados de una empresa utilizando un código numérico como clave de ordenamiento. Este ejercicio aplica los conceptos de jerarquía vistos en el organigrama empresarial, donde se identifican claramente la raíz, los niveles, los nodos internos y las hojas del árbol.

Específicos:

- Implementar un Árbol (BTS) en C++.
- Gestionar inserción, búsqueda, y recorrido en el árbol.
- Modelar un problema del mundo acerca de una jerarquía empresarial mediante estructura de datos.

Contexto del problema

- El juego consiste en defender una ruta o territorio de oleadas de enemigos que avanzan progresivamente hacia una base. Para impedir su avance, el jugador puede colocar estructuras defensivas tales como torres, personajes, edificios u otros elementos de defensa. Estas defensas atacan automáticamente a los enemigos cuando estos ingresan en su rango de alcance.

Instrucciones

Problema práctico

Una empresa necesita registrar empleados en una estructura jerárquica usando un árbol binario de búsqueda.

Cada empleado tendrá:

- Código
- Nombre
- cargo

El árbol debe permitir:

- Insertar empleados.
- Buscar un empleado por código.

- Mostrar recorrido inorden.
- Mostrar recorrido preorden.
- Mostrar recorrido postorden.
- Calcular altura del árbol.
- Identificar hojas.

Mostrar la raíz del árbol.

3. Organización del grupo

Rol	Responsabilidad
Líder	Coordina el avance y revisa el repositorio
Programador 1	Implementa estructura Nodo y función insertar
Programador 2	Implementa búsqueda y recorridos
Programador 3	Implementa altura y hojas
Documentador	Crea README, capturas y explicación
Expositor	Presenta funcionamiento y conclusiones

Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Internet
- Dev – C++

Elementos que debe incluir el programa

Debe implementarse una lista circular para controlar las oleadas del juego y permitir el recorrido continuo entre ellas.

Agregar capturas de:

1. Menú principal

2. Inserción de empleados

3. Búsqueda

4. Recorridos

5. Altura y hojas

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial

Otros (Especifique): _____

Actividades por desarrollar

Implementar un árbol binario de búsqueda en C++ para organizar empleados de una empresa.

Resultados obtenidos

2. Fundamentos Teóricos del BST

Un Árbol Binario de Búsqueda es una estructura de datos jerárquica en la que cada nodo puede tener como máximo dos hijos (izquierdo y derecho). La propiedad fundamental del BST establece que todos los valores en el subárbol izquierdo de un nodo son menores a su clave, y todos los del subárbol derecho son mayores. Esta propiedad permite realizar operaciones de búsqueda, inserción y eliminación de forma eficiente.

2.1 Conceptos clave

Raíz	Es el nodo principal del árbol. No tiene padre. Es el punto de entrada a toda la estructura. En este proyecto, el primer empleado insertado se convierte en la raíz.

Nodo Interno	Todo nodo que tiene al menos un hijo (izquierdo, derecho o ambos). Representa un nivel intermedio de la jerarquía empresarial.

Hoja	Nodo que no tiene ningún hijo (izquierdo == nullptr && derecho == nullptr). Representa el nivel más bajo de la jerarquía.

Nivel	La profundidad de un nodo dentro del árbol. La raíz está en el nivel 1. Sus hijos en el nivel 2, y así sucesivamente.

Altura	La altura del árbol es la longitud del camino más largo desde la raíz hasta una hoja. Se calcula recursivamente como 1 + max(altura_izq, altura_der).

Recorrido Inorden	Visita los nodos en el orden: izquierdo → raíz → derecho. En un BST, produce los elementos en orden ascendente por código.

Recorrido Preorden	Visita los nodos en el orden: raíz → izquierdo → derecho. Útil para copiar o serializar la estructura del árbol.

Recorrido Postorden	Visita los nodos en el orden: izquierdo → derecho → raíz. Utilizado para liberar memoria o procesar dependencias.

3. Descripción del Código Fuente

El código implementado en C++ utiliza una clase ArbolBST que encapsula la lógica del árbol binario de búsqueda. Se compone de dos estructuras de datos y una clase principal.

3.1 Estructura Empleado

Almacena los datos de cada empleado que se registra en el árbol. Contiene tres campos: codigo (entero, clave de búsqueda), nombre (cadena de texto) y cargo (cadena de texto).

```
struct Empleado {
    int codigo;       // Clave del BST
    string nombre;    // Nombre completo del empleado
    string cargo;     // Cargo dentro de la empresa
};
```

3.2 Estructura Nodo

Representa cada nodo del árbol. Contiene un objeto Empleado como dato, y dos punteros hacia los hijos izquierdo y derecho. El constructor inicializa los punteros en nullptr.

```
struct Nodo {
    Empleado dato;
    Nodo* izquierdo;
    Nodo* derecho;
    Nodo(Empleado emp) {
        dato = emp;
        izquierdo = nullptr;
```

```
    derecho = nullptr;

  }

};
```

3.3 Clase ArbolBST — Métodos implementados

Método	Acceso	Descripción
insertar(nodo, emp)	Privado	Inserta recursivamente un empleado en la posición correcta según su código.
buscar(nodo, codigo)	Privado	Busca recursivamente un nodo por código. Retorna el nodo o nullptr.
inorden(nodo)	Privado	Recorre el árbol izquierdo → raíz → derecho e imprime empleados en orden.
preorden(nodo)	Privado	Recorre el árbol raíz → izquierdo → derecho.
postorden(nodo)	Privado	Recorre el árbol izquierdo → derecho → raíz.
altura(nodo)	Privado	Calcula la altura del árbol de forma recursiva usando max().
mostrarHojas(nodo)	Privado	Imprime todos los nodos que no tienen hijos.
mostrarEmpleado(nodo)	Privado	Imprime código, nombre y cargo de un nodo específico.
insertarEmpleado(emp)	Público	Llama al método privado insertar desde la raíz.
buscarEmpleado(codigo)	Público	Busca e imprime el empleado con el código dado.
mostrarRaiz()	Público	Imprime los datos del nodo raíz del árbol.
mostrarInorden()	Público	Imprime el encabezado y llama a inorden(raiz).
mostrarPreorden()	Público	Imprime el encabezado y llama a preorden(raiz).
mostrarPostorden()	Público	Imprime el encabezado y llama a postorden(raiz).

mostrarAltura()	Público	Imprime la altura del árbol llamando a altura(raiz).
mostrarNodosHoja()	Público	Imprime el encabezado y llama a mostrarHojas(raiz).

4. Datos de Prueba y Resultados Esperados

Se utilizan los siguientes empleados para probar el sistema. El primer empleado insertado (código 50) se convierte en la raíz del árbol.

Código	Nombre	Cargo	Tipo de Nodo
50	Empresa UTA	Raíz	Raíz
30	Gerente Ventas	Nodo interno	Nodo Interno
70	Gerente Finanzas	Nodo interno	Nodo Interno
20	Emp 1	Hoja	Hoja
40	Emp 2	Hoja	Hoja
60	Emp 3	Hoja	Hoja
80	Emp 4	Hoja	Hoja

4.1 Árbol resultante

```
        50  (Raíz)
       /   \
     30     70  (Nodos internos)
    / \   / \
   20  40 60  80  (Hojas)
```

4.2 Resultados esperados por operación

Recorrido Inorden (orden ascendente):
20 → 30 → 40 → 50 → 60 → 70 → 80

Recorrido Preorden:
50 → 30 → 20 → 40 → 70 → 60 → 80

Recorrido Postorden:
20 → 40 → 30 → 60 → 80 → 70 → 50

Altura del árbol:
3 (raíz → nodo interno → hoja = 3 niveles)

Nodos hoja:
Emp 1 (20), Emp 2 (40), Emp 3 (60), Emp 4 (80)

Funcionalidades del Sistema (Menú)

El programa presenta un menú interactivo con las siguientes opciones. Cada opción llama al método público correspondiente de la clase ArbolBST.

Opción	Descripción	Método invocado
1	Insertar empleado (código, nombre, cargo)	insertarEmpleado(emp)
2	Buscar empleado por código	buscarEmpleado(codigo)
3	Mostrar nodo raíz del árbol	mostrarRaiz()
4	Recorrido inorden	mostrarInorden()
5	Recorrido preorden	mostrarPreorden()
6	Recorrido postorden	mostrarPostorden()
7	Mostrar altura del árbol	mostrarAltura()
8	Mostrar nodos hoja	mostrarNodosHoja()
0	Salir del programa	(termina el bucle do-while)

Capturas del código

ArbolBinario.cpp U

ArbolBinario.cpp > ...

```cpp
#include <iostream>
#include <string>
#include <algorithm> // Necesario para la función max()

using namespace std;

// Estructura para almacenar los datos del empleado
struct Empleado {
    int codigo;
    string nombre;
    string cargo;
};

// Estructura del Nodo del árbol
struct Nodo {
    Empleado dato;
    Nodo* izquierdo;
    Nodo* derecho;

    // Constructor para inicializar el nodo
    Nodo(Empleado emp) {
        dato = emp;
        izquierdo = nullptr;
        derecho = nullptr;
    }
};

class ArbolBST {
private:
    Nodo* raiz;

    // Función auxiliar para insertar un empleado
    Nodo* insertar(Nodo* nodo, Empleado emp) {
        if (nodo == nullptr) {
            return new Nodo(emp);
        }
        if (emp.codigo < nodo->dato.codigo) {
            nodo->izquierdo = insertar(nodo->izquierdo, emp);
        } else if (emp.codigo > nodo->dato.codigo) {
            nodo->derecho = insertar(nodo->derecho, emp);
        } else {
            cout << "El codigo ya existe.\n";
        }
        return nodo;
    }

    // Función auxiliar para buscar por código
    Nodo* buscar(Nodo* nodo, int codigo) {
        if (nodo == nullptr || nodo->dato.codigo == codigo) {
            return nodo;
        }
        if (codigo < nodo->dato.codigo) {
            return buscar(nodo->izquierdo, codigo);
        } else {
            return buscar(nodo->derecho, codigo);
        }
    }

    // Recorridos del árbol
    void inorden(Nodo* nodo) {
        if (nodo != nullptr) {
            inorden(nodo->izquierdo);
            mostrarEmpleado(nodo);
```

```
            inorden(nodo->derecho);
        }
    }

    void preorden(Nodo* nodo) {
        if (nodo != nullptr) {
            mostrarEmpleado(nodo);
            preorden(nodo->izquierdo);
            preorden(nodo->derecho);
        }
    }

    void postorden(Nodo* nodo) {
        if (nodo != nullptr) {
            postorden(nodo->izquierdo);
            postorden(nodo->derecho);
            mostrarEmpleado(nodo);
        }
    }

    // Cálculo de la altura del árbol
    int altura(Nodo* nodo) {
        if (nodo == nullptr) {
            return 0;
        }
        int alturaIzq = altura(nodo->izquierdo);
        int alturaDer = altura(nodo->derecho);
        return 1 + max(alturaIzq, alturaDer);
    }

    // Función para identificar y mostrar hojas
    void mostrarHojas(Nodo* nodo) {
        if (nodo != nullptr) {
            if (nodo->izquierdo == nullptr && nodo->derecho == nullptr) {
                mostrarEmpleado(nodo);
            }
            mostrarHojas(nodo->izquierdo);
            mostrarHojas(nodo->derecho);
        }
    }

    void mostrarEmpleado(Nodo* nodo) {
        cout << "Codigo: " << nodo->dato.codigo
             << " | Nombre: " << nodo->dato.nombre
             << " | Cargo: " << nodo->dato.cargo << endl;
    }

public:
    ArbolBST() {
        raiz = nullptr;
    }

    // Métodos públicos para interactuar con el árbol
    void insertarEmpleado(Empleado emp) { raiz = insertar(raiz, emp); }

```

```
    void buscarEmpleado(int codigo) {
        Nodo* resultado = buscar(raiz, codigo);
        if (resultado != nullptr) {
            cout << "\nEmpleado encontrado:\n";
            mostrarEmpleado(resultado);
        } else {
            cout << "\nEmpleado no encontrado.\n";
        }
    }

    void mostrarRaiz() {
        if (raiz != nullptr) {
            cout << "\nRaiz del arbol:\n";
            mostrarEmpleado(raiz);
        } else {
            cout << "El arbol esta vacio.\n";
        }
    }

    void mostrarInorden() { cout << "\nRecorrido Inorden:\n"; inorden(raiz); }
    void mostrarPreorden() { cout << "\nRecorrido Preorden:\n"; preorden(raiz); }
    void mostrarPostorden() { cout << "\nRecorrido Postorden:\n"; postorden(raiz); }
    void mostrarAltura() { cout << "\nAltura del arbol: " << altura(raiz) << endl; }
    void mostrarNodosHoja() { cout << "\nNodos hoja:\n"; mostrarHojas(raiz); }
};

int main() {
    ArbolBST arbol;
    int opcion;
    do {
        cout << "\n===== MENU ARBOL BST EMPRESARIAL =====\n";
        cout << "1. Insertar empleado\n2. Buscar empleado\n3. Mostrar raiz\n";
        cout << "4. Recorrido inorden\n5. Recorrido preorden\n6. Recorrido postorden\n";
        cout << "7. Mostrar altura\n8. Mostrar hojas\n0. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        if (opcion == 1) {
            Empleado emp;
            cout << "Codigo: "; cin >> emp.codigo;
            cin.ignore();
            cout << "Nombre: "; getline(cin, emp.nombre);
            cout << "Cargo: "; getline(cin, emp.cargo);
            arbol.insertarEmpleado(emp);
        } else if (opcion == 2) {
            int cod; cout << "Ingrese codigo a buscar: "; cin >> cod;
            arbol.buscarEmpleado(cod);
        } else if (opcion == 3) {
            arbol.mostrarRaiz();
        } else if (opcion == 4) {
            arbol.mostrarInorden();
        } else if (opcion == 5) {
            arbol.mostrarPreorden();
        } else if (opcion == 6) {
            arbol.mostrarPostorden();
        } else if (opcion == 7) {
            arbol.mostrarAltura();
        } else if (opcion == 8) {
            arbol.mostrarNodosHoja();
        }
    } while (opcion != 0);

    return 0;
}
```

Habilidades blandas empleadas en la prueba práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

Conclusiones

El Árbol Binario de Búsqueda es una estructura de datos fundamental en Ciencias de la Computación que permite organizar información jerárquica de manera eficiente. En este proyecto, se aplicó el concepto de BST para modelar un organigrama empresarial, donde cada empleado ocupa una posición determinada por su código numérico.

La propiedad de ordenamiento del BST garantiza que el recorrido inorden produzca los empleados en orden ascendente por código. La complejidad de búsqueda e inserción es O(log n) en el caso promedio para árboles balanceados, lo que hace al BST una alternativa eficiente frente a estructuras lineales como arreglos o listas enlazadas.

La implementación en C++ con punteros y recursividad refuerza el manejo de memoria dinámica y el paradigma de programación orientada a objetos mediante el encapsulamiento de la lógica dentro de la clase ArbolBST. El uso de métodos privados auxiliares y métodos públicos de interfaz constituye una práctica de diseño limpio y modular.

Recomendaciones

Se recomienda extender la implementación actual del Árbol Binario de Búsqueda incorporando la función de eliminación de nodos, ya que constituye una operación fundamental que completa el conjunto de operaciones básicas del BST y representa un mayor nivel de complejidad al requerir el manejo de tres casos distintos: eliminación de una hoja, de un nodo con un solo hijo y de un nodo con dos hijos. Adicionalmente, sería beneficioso explorar estructuras de árboles autobalanceados como el AVL o el árbol Rojo-Negro, los cuales garantizan una altura mínima en todo momento y evitan la degradación del rendimiento a O(n) que puede ocurrir en un BST convencional cuando los datos se insertan en orden ascendente o descendente, asegurando así búsquedas eficientes independientemente del orden de inserción.

Anexos

Link Github: https://github.com/joel427/Deberes---Estructura-de-datos/upload/main

Resultado final del Árbol empresarial

Registro.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
Codigo: 50
Nombre: Empresa UTA
Cargo: Raiz
```

Búsqueda por código

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 80

Empleado encontrado:
Codigo: 80 | Nombre: Emp 4 | Cargo: hoja
```

Mostrar raíz

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 3

Raiz del arbol:
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
```

Recorrido en orden

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 4

Recorrido Inorden:
Codigo: 20 | Nombre: Emp 1 | Cargo: hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 40 | Nombre: Emp 2 | Cargo: hoja
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 60 | Nombre: Emp 3 | Cargo: hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 80 | Nombre: Emp 4 | Cargo: hoja
```

Recorrido de pre orden

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 5

Recorrido Preorden:
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 20 | Nombre: Emp 1 | Cargo: hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp 3 | Cargo: hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: hoja
```

Recorrido Postorden

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 6

Recorrido Postorden:
Codigo: 20 | Nombre: Emp 1 | Cargo: hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: hoja
Codigo: 30 | Nombre: Gerente Ventas | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp 3 | Cargo: hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: hoja
Codigo: 70 | Nombre: Gerente Finanzas | Cargo: Nodo interno
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
```

Altura del Árbol

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 7

Altura del arbol: 3
```

Mostrar Hoja

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 8

Nodos hoja:
Codigo: 20 | Nombre: Emp 1 | Cargo: hoja
Codigo: 40 | Nombre: Emp 2 | Cargo: hoja
Codigo: 60 | Nombre: Emp 3 | Cargo: hoja
Codigo: 80 | Nombre: Emp 4 | Cargo: hoja
```